
Pruebas con xUnit y .NET 10

Pruebas unitarias y de integración para C#, Razor Pages, EF Core y API
ASP.NET Core

Finalidad

Dirigido a alumnado que empieza a verificar programas C# de manera automática y necesita avanzar desde una primera prueba hasta una suite mantenible para aplicaciones Razor Pages y API.

Al terminar este manual podrás:

- Distinguir pruebas unitarias, de integración y de extremo a extremo.
- Escribir pruebas legibles con Fact, Theory, datos parametrizados y aserciones precisas.
- Probar código asíncrono, excepciones, servicios, PageModel y validación.
- Aislar EF Core con SQLite en memoria y probar la aplicación con WebApplicationFactory.
- Usar fixtures, salida de diagnóstico, filtros, cobertura y Microsoft Testing Platform.
- Evitar pruebas frágiles, lentas o acopladas a detalles internos.

Edición didáctica 2026 · .NET 10

Índice

1. Qué debe demostrar una prueba	4
Pirámide de pruebas	4
Propiedades FIRST	4
2. Crear un proyecto xUnit v3 para net10.0	5
Estructura de solución	5
Ejecutar	5
3. La primera prueba: Arrange, Act, Assert	7
Nombres que explican	7
4. Fact, Theory y datos parametrizados	8
Fact	8
Theory e InlineData	8
MemberData para objetos complejos	8
5. Aserciones precisas	9
6. Excepciones, async y cancelación	10
Excepciones síncronas	10
Pruebas asíncronas	10
Tiempo y cancelación	10
7. Diseño comprobable y dobles de prueba	11
Falso, stub, mock y spy	11
Fronteras a controlar	11
8. Ciclo de vida y fixtures	12
Ámbitos	12
9. Probar EF Core con SQLite en memoria	13

10. Probar PageModel y validación	14
11. Pruebas de integración con WebApplicationFactory	15
Hacer visible Program	15
Proyecto de pruebas	15
Fábrica personalizada	15
12. Pruebas de API completas	17
Qué comprobar	17
No pruebes una API externa en tu suite normal	17
13. Paralelismo, categorías y salida	18
Salida diagnóstica	18
14. Cobertura, MTP y automatización	19
Cobertura	19
Pipeline recomendado	19
15. Errores frecuentes y revisión de una suite	20
Checklist de revisión	20
16. Laboratorio progresivo	21
Rúbrica breve	21
17. Referencias	22

1. Qué debe demostrar una prueba

Una prueba automatizada prepara un escenario, ejecuta una unidad de comportamiento y comprueba un resultado observable. Su valor no está en aumentar un porcentaje, sino en detectar regresiones y describir con precisión qué promete el código.

Pirámide de pruebas

Nivel	Alcance	Velocidad	Ejemplo
Unitaria	una regla o clase aislada	muy alta	calcular puntuación
Integración	componentes reales colaborando	media	GET /api/preguntas con EF
Navegador	sistema visible para el usuario	menor	crear y borrar desde la interfaz

Propiedades FIRST

- **Fast:** rápidas para ejecutarlas continuamente.
- **Independent:** el resultado no depende del orden ni de otra prueba.
- **Repeatable:** mismas entradas, mismo resultado en cualquier equipo adecuado.
- **Self-validating:** termina en aprobado o fallido sin inspección manual.
- **Timely:** se escribe cerca del comportamiento que protege.

Unidad no significa método

La unidad útil es un comportamiento. Una prueba puede atravesar varios métodos internos sin perder su carácter unitario si no cruza fronteras lentas o incontrolables.

2. Crear un proyecto xUnit v3 para net10.0

Estructura de solución

BASH

```
1 dotnet new sln -n Trivial
2 dotnet new webapp -n TrivialApi
3 dotnet new xunit -n TrivialApi.Tests
4
5 dotnet sln add TrivialApi/TrivialApi.csproj
6 dotnet sln add TrivialApi.Tests/TrivialApi.Tests.csproj
7 dotnet add TrivialApi.Tests reference TrivialApi
8 dotnet test
```

El template instalado puede evolucionar con el SDK. Para usar explícitamente xUnit v3, el proyecto de pruebas referencia **xunit.v3**. xUnit v3 produce un ejecutable de pruebas y admite tanto su corredor integrado como Microsoft Testing Platform. Conviene mantener el adaptador y Microsoft.NET.Test.Sdk mientras todos los entornos del curso no estén alineados con MTP.

HTML

```
1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <TargetFramework>net10.0</TargetFramework>
4     <IsPackable>false</IsPackable>
5     <IsTestProject>true</IsTestProject>
6     <Nullable>enable</Nullable>
7   </PropertyGroup>
8
9   <ItemGroup>
10    <PackageReference Include="xunit.v3" />
11    <PackageReference Include="xunit.runner.visualstudio" />
12    <PackageReference Include="Microsoft.NET.Test.Sdk" />
13  </ItemGroup>
14
15  <ItemGroup>
16    <ProjectReference Include="../TrivialApi/TrivialApi.csproj" />
17  </ItemGroup>
18 </Project>
```

Ejecutar

BASH

```
1 dotnet test
2 dotnet test --no-restore
3 dotnet test --filter "FullyQualifiedName~CalculadoraTests"
4 dotnet test --logger "console;verbosity=detailed"
5
6 # xUnit v3 también puede ejecutarse como aplicación.
7 dotnet run --project TrivialApi.Tests
```

3. La primera prueba: Arrange, Act, Assert

```
CSHARP
1 public sealed class CalculadoraTests
2 {
3     [Fact]
4     public void Sumar_DosNumeros_DevuelveLaSuma()
5     {
6         // Arrange
7         var calculadora = new Calculadora();
8
9         // Act
10        int resultado = calculadora.Sumar(2, 3);
11
12        // Assert
13        Assert.Equal(5, resultado);
14    }
15 }
```

Arrange prepara; Act ejecuta una sola acción principal; Assert comprueba el resultado. Los comentarios pueden omitirse cuando las tres fases se ven con claridad, pero ayudan al comenzar.

Nombres que explican

Patrón	Ejemplo
Método_Escenario_Resultado	Calcular_RespuestaIncorrecta_NoSumaPunto
Debería...Cuando...	DeberiaRechazarCantidadCuandoEsNegativa
Frase de comportamiento	UnaCategoriaSinNombreEsInvalida

Una causa de fallo

Una prueba debe fallar por una razón reconocible. Varias aserciones son aceptables si describen una sola conducta; si mezclan escenarios, sepáralas.

4. Fact, Theory y datos parametrizados

Fact

Fact representa un caso sin parámetros externos. Debe ser determinista: no depende de la hora real, una web externa ni datos compartidos.

Theory e InlineData

```
CSHARP
1  [Theory]
2  [InlineData(0, 1)]
3  [InlineData(1, 1)]
4  [InlineData(2, 2)]
5  [InlineData(5, 120)]
6  public void Factorial_ValoresValidos_DevuelveResultado(
7      int numero,
8      int esperado)
9  {
10     int actual = Matematicas.Factorial(numero);
11     Assert.Equal(esperado, actual);
12 }
```

MemberData para objetos complejos

```
CSHARP
1  public static TheoryData<Pregunta, string, bool> CasosRespuesta => new()
2  {
3      { new Pregunta { RespuestaCorrecta = "París" }, "París", true },
4      { new Pregunta { RespuestaCorrecta = "París" }, "Londres", false },
5  };
6
7  [Theory]
8  [MemberData(nameof(CasosRespuesta))]
9  public void ComprobarRespuesta_DevuelveLoEsperado(
10     Pregunta pregunta,
11     string respuesta,
12     bool esperado)
13  {
14     Assert.Equal(esperado, pregunta.EsCorrecta(respuesta));
15 }
```

ClassData es útil cuando los casos merecen una clase reutilizable. No conviertas una Theory en una tabla inmanejable: si un caso necesita una preparación o una explicación distinta, merece un Fact propio.

5. Aserciones precisas

Intención	Aserción
igualdad de valor	Assert.Equal(esperado, actual)
mismo objeto	Assert.Same(esperado, actual)
nulo/no nulo	Assert.Null / Assert.NotNull
booleano	Assert.True / Assert.False
cadena contiene	Assert.Contains(fragmento, texto)
colección	Assert.Empty, Single, All, Collection
tipo	Assert.IsType / IsAssignableFrom
intervalo	Assert.InRange(actual, minimo, maximo)

CSHARP

```
1 Assert.Collection(  
2     resultado,  
3     primera => Assert.Equal("Arte", primera.Nombre),  
4     segunda => Assert.Equal("Ciencia", segunda.Nombre));  
5  
6 Categoria unica = Assert.Single(resultado);  
7 Assert.StartsWith("C", unica.Nombre);
```

Mensaje de fallo

Usa la aserción más específica. `Assert.True(lista.Count == 1)` ofrece menos información que `Assert.Single(lista)`. Un buen fallo reduce el tiempo de diagnóstico.

6. Excepciones, async y cancelación

Excepciones síncronas

CSHARP

```
1 [Fact]
2 public void Factorial_NumeroNegativo_LanzaExcepcion()
3 {
4     var excepcion = Assert.Throws<ArgumentOutOfRangeException>(
5         () => Matematicas.Factorial(-1));
6
7     Assert.Equal("numero", excepcion.ParamName);
8 }
```

Pruebas asíncronas

CSHARP

```
1 [Fact]
2 public async Task ObtenerAsync_IdInexistente_DevuelveNull()
3 {
4     var repositorio = new RepositorioFalso();
5     var servicio = new ServicioPreguntas(repositorio);
6
7     Pregunta? resultado = await servicio.ObtenerAsync(999);
8
9     Assert.Null(resultado);
10 }
11
12 [Fact]
13 public async Task GuardarAsync_DatoInvalido_LanzaExcepcion()
14 {
15     await Assert.ThrowsAsync<ValidationException>(
16         () => servicio.GuardarAsync(new EntradaPregunta()));
17 }
```

El método de prueba devuelve Task y espera la operación. No uses async void: el corredor no puede observar correctamente su finalización y excepciones.

Tiempo y cancelación

Para probar lógica temporal, inyecta TimeProvider en lugar de llamar directamente a DateTime.Now. Para cancelación, usa un CancellationTokenSource controlado y verifica el resultado que tu contrato promete, normalmente OperationCanceledException o TaskCanceledException.

7. Diseño comprobable y dobles de prueba

Falso, stub, mock y spy

Doble	Función
Fake	implementación funcional simplificada, por ejemplo un repositorio en memoria
Stub	devuelve respuestas preparadas
Spy	registra llamadas para consultarlas después
Mock	verifica interacciones esperadas, a menudo mediante biblioteca

```
CSHARP
1 public sealed class RepositorioPreguntasFalso : IRepositoryPreguntas
2 {
3     public List<Pregunta> Datos { get; } = [];
4
5     public Task AgregarAsync(
6         Pregunta pregunta,
7         CancellationToken cancellationToken = default)
8     {
9         Datos.Add(pregunta);
10        return Task.CompletedTask;
11    }
12 }
```

Prefiere comprobar resultados y estado observable. Verificar cada llamada interna acopla la prueba a la implementación y dificulta refactorizar. Un mock es útil cuando la interacción es el comportamiento: por ejemplo, no enviar dos correos o registrar una operación crítica.

Fronteras a controlar

- Hora: TimeProvider.
- Aleatoriedad: interfaz o Random con semilla conocida.
- HTTP: HttpResponseMessage falso o servidor de pruebas.
- Archivos: abstracción pequeña o directorio temporal por prueba.
- Base de datos: SQLite aislado para semántica relacional.

8. Ciclo de vida y fixtures

xUnit crea una instancia nueva de la clase de prueba para cada caso. El constructor prepara contexto barato por prueba. IDisposable o IAsyncDisposable limpian recursos. IAsyncLifetime permite preparación asíncrona.

```
CSHARP
1 public sealed class BaseDatosFixture : IAsyncLifetime
2 {
3     public SQLiteConnection Conexion { get; private set; } = null!;
4
5     public async ValueTask InitializeAsync()
6     {
7         Conexion = new SQLiteConnection("Data Source=:memory:");
8         await Conexion.OpenAsync();
9     }
10
11    public async ValueTask DisposeAsync()
12    {
13        await Conexion.DisposeAsync();
14    }
15 }
16
17 public sealed class CategoriaTests(BaseDatosFixture fixture)
18     : IClassFixture<BaseDatosFixture>
19 {
20     // La fixture se comparte entre los tests de esta clase.
21 }
```

Ámbitos

Mecanismo	Se comparte entre
Constructor de la clase	no comparte instancia; se ejecuta por prueba
IClassFixture	todas las pruebas de una clase
Collection fixture	varias clases de una colección
Assembly fixture v3	todo el ensamblado de pruebas

Aislamiento primero

Compartir una fixture cara puede ahorrar tiempo, pero no debe compartir datos mutables entre pruebas. Reinicia el estado o asigna recursos independientes.

9. Probar EF Core con SQLite en memoria

El proveedor InMemory de EF Core no reproduce claves, transacciones ni SQL relacional. Para comprobar consultas y restricciones de una aplicación SQLite, usa SQLite en memoria manteniendo la conexión abierta durante la prueba.

CSHARP

```
1 static async Task<(SqlConnection, TrivialContext)> CrearContextoAsync()
2 {
3     var conexion = new SqlConnection("Data Source=:memory:");
4     await conexion.OpenAsync();
5
6     var opciones = new DbContextOptionsBuilder<TrivialContext>()
7         .UseSqlite(conexion)
8         .Options;
9
10    var contexto = new TrivialContext(opciones);
11    await contexto.Database.EnsureCreatedAsync();
12    return (conexion, contexto);
13 }
```

CSHARP

```
1 [Fact]
2 public async Task NombreRepetido_IndiceUnico_RechazaSegundoRegistro()
3 {
4     var (conexion, contexto) = await CrearContextoAsync();
5     await using var _ = conexion;
6     await using var __ = contexto;
7
8     contexto.Categorias.Add(new() { Nombre = "Ciencia" });
9     await contexto.SaveChangesAsync();
10
11    contexto.Categorias.Add(new() { Nombre = "Ciencia" });
12
13    await Assert.ThrowsAsync<DbUpdateException>(
14        () => contexto.SaveChangesAsync());
15 }
```

- Cada prueba debe crear datos explícitos y no depender de una base .db del proyecto.
- No reutilices la misma conexión en pruebas paralelas sin aislamiento.
- Prueba las consultas importantes contra el mismo proveedor usado en producción cuando sea viable.

10. Probar PageModel y validación

CSHARP

```
1 [Fact]
2 public async Task OnPostAsync_ModeloInvalido_DevuelvePagina()
3 {
4     var servicio = new ServicioPreguntasFalso();
5     var model = new CrearModel(servicio);
6     model.ModelState.AddModelError("Entrada.Texto", "Obligatorio");
7
8     IActionResult resultado = await model.OnPostAsync();
9
10    Assert.IsType<PageResult>(resultado);
11    Assert.Empty(servicio.Guardadas);
12 }
```

CSHARP

```
1 [Fact]
2 public async Task OnPostAsync_EntradaValida_GuardaYRedirige()
3 {
4     var servicio = new ServicioPreguntasFalso();
5     var model = new CrearModel(servicio)
6     {
7         Entrada = new() { Texto = "¿Capital de Francia?", CategoriaId = 1 }
8     };
9
10    IActionResult resultado = await model.OnPostAsync();
11
12    var redireccion = Assert.IsType<RedirectToPageResult>(resultado);
13    Assert.Equal("./Index", redireccion.PageName);
14    Assert.Single(servicio.Guardadas);
15 }
```

DataAnnotations no se ejecuta sola

Al invocar directamente un PageModel no pasa por model binding. Si quieres comprobar atributos de validación, ejecuta `Validator.TryValidateObject` o usa una prueba de integración que atraviese el pipeline.

11. Pruebas de integración con WebApplicationFactory

Hacer visible Program

CSHARP

```
1 // Al final de Program.cs
2 public partial class Program { }
```

Proyecto de pruebas

BASH

```
1 dotnet add TrivialApi.Tests package Microsoft.AspNetCore.Mvc.Testing
```

CSHARP

```
1 public sealed class ApiTests(
2     WebApplicationFactory<Program> factory)
3     : IClassFixture<WebApplicationFactory<Program>>
4 {
5     [Fact]
6     public async Task GetPreguntas_DevuelveRespuestaCorrecta()
7     {
8         HttpClient cliente = factory.CreateClient();
9
10        HttpResponseMessage respuesta =
11            await cliente.GetAsync("/api/preguntas?cantidad=5");
12
13        respuesta.EnsureSuccessStatusCode();
14        var preguntas = await respuesta.Content
15            .ReadFromJsonAsync<List<PreguntaApiDto>>();
16
17        Assert.NotNull(preguntas);
18        Assert.InRange(preguntas.Count, 0, 5);
19    }
20 }
```

Fábrica personalizada

CSHARP

```
1 public sealed class FabricaApi : WebApplicationFactory<Program>
2 {
3     protected override void ConfigureWebHost(IWebHostBuilder builder)
4     {
5         builder.UseEnvironment("Testing");
6
7         builder.ConfigureServices(services =>
8         {
9             services.RemoveAll<DbContextOptions<TrivialContext>>();
10            services.AddDbContext<TrivialContext>(options =>
11                options.UseSqlite("Data Source=:memory:"));
12            });
13     }
14 }
```

El nombre `FabricaApi` no tiene significado especial: describe una clase que crea la aplicación bajo prueba y sustituye configuración o servicios. El servidor de pruebas atraviesa middleware, routing, binding, filtros y serialización sin abrir un puerto real.

Base de datos en memoria

Una cadena `Data Source=:memory:` crea una base por conexión. Si el contexto abre y cierra conexiones, los datos desaparecen. La fábrica debe conservar una conexión abierta o usar un archivo temporal único por ejecución.

12. Pruebas de API completas

Qué comprobar

- Código de estado y Content-Type.
- Esquema y valores JSON importantes, no el texto JSON completo si el orden no es contractual.
- Límites de parámetros, filtros y paginación.
- 404, 400, 401, 403 y conflictos esperados.
- Persistencia posterior al POST/PUT/DELETE.
- CORS solo cuando el contrato de navegador lo requiere.

CSHARP

```
1  [Theory]
2  [InlineData(0)]
3  [InlineData(-1)]
4  [InlineData(1001)]
5  public async Task CantidadFueraDeRango_Selimita(int cantidad)
6  {
7      HttpClient cliente = _factory.CreateClient();
8
9      var respuesta = await cliente.GetFromJsonAsync<List<PreguntaApiDto>>("
10         $"/api/preguntas?cantidad={cantidad}");
11
12      Assert.NotNull(respuesta);
13      Assert.InRange(respuesta.Count, 0, 100);
14 }
```

No pruebes una API externa en tu suite normal

Una prueba que llama a NASA o PokeAPI puede fallar por red, cuota o cambios ajenos. Simula la frontera HTTP para pruebas deterministas y reserva smoke tests externos, etiquetados y opcionales, para comprobar integración real.

13. Paralelismo, categorías y salida

xUnit ejecuta pruebas de colecciones diferentes en paralelo. Esto acelera la suite, pero descubre errores si las pruebas comparten puertos, archivos, base de datos o variables globales.

```
CSHARP
1  [CollectionDefinition("Base de datos", DisableParallelization = true)]
2  public sealed class ColeccionBaseDatos
3      : ICollectionFixture<BaseDatosFixture>
4  {
5  }
6
7  [Collection("Base de datos")]
8  public sealed class PreguntasIntegracionTests
9  {
10 }
```

Salida diagnóstica

```
CSHARP
1  public sealed class ConsultaTests(ITestOutputHelper output)
2  {
3      [Fact]
4      public void MuestraContextoUtil()
5      {
6          output.WriteLine("Semilla usada: {0}", 12345);
7          Assert.True(true);
8      }
9  }
```

La salida debe ayudar a reproducir un fallo, no inundar la consola. En xUnit v3 también puede adjuntarse un archivo mediante `TestContext` cuando el corredor lo admite.

14. Cobertura, MTP y automatización

Cobertura

La cobertura indica qué código se ejecutó, no si se comprobó bien. Una línea cubierta sin una aserción significativa puede seguir siendo incorrecta. Úsala para descubrir huecos, especialmente ramas de error, no como objetivo aislado.

BASH

```
1 dotnet test --collect:"XPlat Code Coverage"
2
3 # Con xUnit v3 y Microsoft Testing Platform, consulta la
4 # configuración de cobertura compatible con la versión instalada.
5 dotnet test --help
```

Pipeline recomendado

1. Restaurar con bloqueo o versiones centralizadas.
2. Compilar Release tratando las advertencias acordadas como errores.
3. Ejecutar pruebas unitarias e integración con resultados de prueba.
4. Generar cobertura y artefactos diagnósticos.
5. Ejecutar Playwright tras arrancar la aplicación o con fixture web.
6. Publicar solo si todas las puertas de calidad pasan.

.NET 10

El SDK de .NET 10 integra Microsoft Testing Platform en dotnet test. xUnit v3 puede utilizar su corredor propio o MTP. Mantén una estrategia única en clase y documenta el comando exacto que se espera.

15. Errores frecuentes y revisión de una suite

Problema	Corrección
Carácter \ inesperado al principio	el archivo contiene secuencias literales copiadas; abrirlo y eliminar barras/escapes espurios
Prueba depende del orden	crear y limpiar sus propios datos
Puerto ocupado	usar TestServer o asignación dinámica
Program inaccesible	añadir public partial class Program o InternalsVisibleTo
Base en memoria vacía	mantener abierta la conexión SQLite
Nullable genera ruido	inicializar campos de fixture y usar Assert.NotNull antes de acceder
Prueba async pasa demasiado pronto	devolver Task y await
Flaky	eliminar hora/red/aleatoriedad compartida y esperas fijas

Checklist de revisión

- El nombre describe escenario y resultado.
- El fallo señala una conducta, no un detalle de implementación.
- La prueba no lee datos ni configuración del equipo del alumno.
- No hay Thread.Sleep ni dependencia de un puerto fijo.
- Los recursos se liberan incluso si la aserción falla.
- La suite completa puede repetirse varias veces con el mismo resultado.

16. Laboratorio progresivo

1. Prueba una clase Puntuacion con Fact para acierto y fallo.
2. Convierte varios casos límite en Theory con InlineData.
3. Añade excepción para cantidad negativa y prueba su ParamName.
4. Extrae IRepositoryPreguntas y crea un fake manual.
5. Prueba un PageModel: ModelState inválido, entidad inexistente y redirección correcta.
6. Crea SQLite en memoria y prueba índice único y relación obligatoria.
7. Añade WebApplicationFactory y comprueba GET de página y API JSON.
8. Configura una fábrica con datos semilla por prueba y evita interferencias paralelas.
9. Ejecuta cobertura y escribe una prueba útil para una rama sin cubrir.
10. Integra la suite en un pipeline y conserva resultados de test.

Rúbrica breve

Nivel	Evidencia
Básico	Fact claros, AAA y aserciones precisas
Intermedio	Theory, async, dobles manuales y aislamiento de datos
Avanzado	WebApplicationFactory, EF realista, fixtures y pipeline estable

17. Referencias

[xUnit.net v3: primeros pasos](#)

[Novedades de xUnit.net v3](#)

[Contexto compartido y fixtures](#)

[xUnit v3 y Microsoft Testing Platform](#)

[Pruebas de integración en ASP.NET Core](#)

[Pruebas de Minimal API](#)